Departamento Ingeniería en Sistemas

Facultad de Informática

Universidad Nacional del Comahue

Materia: Frameworks e Interoperabilidad

Trabajo Práctico Nº3: Servicios Web e Interoperabilidad

Autores

Matias E. Bacsay - Legajo: FAI 4078
Alejo A. Lopez - Legajo: FAI 3146

Cátedra

Marcos Cruz
Jorge Navarro
Lara Acuña Bravo

Octubre 2026

Índice

Índice **1**
Introducción **2**
Desarrollo **3**
Introducción al Protocolo SOAP 3
Arquitectura 3
Componentes principales 4
Mensaje XML 4
WSDL 5
XML Schema (XSD) 5
WSDL (Web Service Description Language) 5
Operaciones 5
Mensajes 5
Tipos de datos 6
Endpoints 6
UDDI (Universal Description, Discovery and Integration) 7
Modelo de datos 7
Relación entre UDDI, WSDL y SOAP 7
Aspectos de seguridad del protocolo 8
WS-Security 9
HTTPS y TLS 10
Firma digital XML Signature 10
Cifrado mediante XML Encryption 11
Comparativa entre SOAP y REST 11
Estándar de lenguaje SOAP 12
Estándar de lenguaje REST 12
¿REST más liviano que SOAP? 13
Estándares más utilizados 14
Ejemplo de caso de consumo 15
Dominios de aplicación exclusiva de SOAP frente a REST 16
Integración de Servicios Web y APIs RESTful Externas (Actividad 2) 17
1. API Geográfica de Argentina (Georef AR) 17
2. Almacenamiento en la Nube (Cloudinary REST API) 17
Uso de IA 18
Conclusión 19

Introducción

La interoperabilidad es la capacidad de dos o más sistemas de intercambiar información y utilizar la información intercambiada. En un contexto donde las aplicaciones se desarrollan con lenguajes, plataformas y dispositivos distintos, los servicios web se consolidaron como el mecanismo principal para lograrla, ya que permiten que un sistema exponga datos y funcionalidades a otros a través de una red, mediante protocolos y estándares comunes.

Este trabajo práctico aborda los dos enfoques más difundidos para construir y consumir servicios web: SOAP y REST. En primer lugar, se investiga el protocolo SOAP, su arquitectura, sus componentes (WSDL, XML Schema y UDDI) y sus aspectos de seguridad, y se lo compara con REST en cuanto a estándares, peso y características. Luego se analiza un caso real de consumo de un servicio SOAP, la facturación electrónica de ARCA (ex AFIP), y se fundamentan dominios donde SOAP resulta más adecuado que REST. Por último, se documenta la integración de dos APIs RESTful externas, Georef AR y Cloudinary, en la plataforma RedAT del trabajo final, un sistema de acompañamiento terapéutico, junto con la justificación de cada elección frente a otras alternativas y el detalle del uso de IA.

Desarrollo

Introducción al Protocolo SOAP

SOAP (Simple Object Access Protocol) es un protocolo estándar de comunicación que permite a dos aplicaciones intercambiar información estructurada, independientemente del lenguaje y la plataforma con la que se haya desarrollado. El formato de los mensajes intercambiados utilizan el formato XML (Extensible Markup Language) para organizar los datos a compartir mediante etiquetas, similar a HTML.

Define las reglas para procesar esta información, funcionando como un contrato formal entre el cliente y el servidor. Los datos suelen transportarse sobre HTTP, aunque puede trabajarse sobre otros como SMTP y TCP.

Arquitectura

La arquitectura de una comunicación SOAP se puede entender a partir de los elementos que participan en el intercambio de mensajes y de las reglas que permiten que dichos elementos se comuniquen. Podemos dividir el servicio en dos actores principales.

- El cliente → Es la aplicación que solicita ejecutar una operación.
- El proveedor → Es la aplicación que ofrece una funcionalidad y recibe las solicitudes que vienen del consumidor.

Para que la comunicación sea posible y funcione correctamente, ambas partes deben respetar la estructura de los mensajes y las reglas que fueron definidas por el servicio. Se debería seguir el siguiente flujo.

1. El cliente identifica el servicio que necesita utilizar, conoce la operación disponible y el formato de los datos que tiene que enviar.
2. Construye el mensaje SOAP con la operación solicitada y los datos necesarios para ejecutarla.
3. El mensaje se transmite al proveedor utilizando el protocolo de transporte.

4. Recibe el mensaje y el procesador SOAP interpreta su estructura para que la aplicación ejecute la lógica correspondiente.

5. El proveedor genera una respuesta, que puede contener los datos solicitados o información sobre un error.

6. La respuesta se devuelve al cliente mediante otro mensaje SOAP, que será interpretado por la aplicación que realizó la solicitud.

Componentes principales

Mensaje XML

Es el mensaje que se transmite entre las aplicaciones, con una estructura definida. Posee los siguientes elementos dentro de su estructura.

- Envelope → Es el elemento que delimita el mensaje SOAP y establece su estructura general. Contiene los elementos que forman parte del mensaje y permite identificar que el documento XML sigue el formato definido por SOAP.

- Header → Es un elemento opcional que permite incluir información adicional necesaria para procesar el mensaje. Puede utilizarse para incorporar datos de seguridad, información de autenticación o metadatos relacionados con la comunicación.

- Body → Es el elemento que contiene la información principal del mensaje. Ahí se representa la operación que se solicita por parte del cliente o los datos que devuelve el servicio.

- Fault → Es el mecanismo definido por SOAP para comunicar información sobre errores producidos durante el procesamiento de un mensaje. Por ejemplo, el servicio podría devolver un error ante una solicitud mal formada o si es incapaz de procesar dicha petición.

WSDL

Es un lenguaje que permite documentar las operaciones disponibles, los mensajes que se intercambian, los tipos de datos utilizados, los protocolos y formatos de comunicación, y las direcciones donde se encuentra disponible el servicio.

XML Schema (XSD)

Permite definir la estructura y los tipos de datos que pueden utilizarse en los mensajes. Especificando elementos, campos, tipos y restricciones que deben respetarse al intercambiar información.

WSDL (Web Service Description Language)

Es un lenguaje basado en XML que permite describir formalmente las características de un servicio web. Su objetivo principal es definir un contrato técnico entre el proveedor del servicio y las aplicaciones que desean consumirlo.
Para entenderlo mejor, se detallan los elementos principales de este lenguaje.

Operaciones

Las operaciones representan las acciones que un servicio ofrece a sus consumidores. Cada operación define una funcionalidad y establece qué mensajes intervienen en su ejecución. Pueden referenciar mensajes de entrada y salida, así como contemplar mensajes de error cuando corresponda.
Por ejemplo, el servicio del sistema de acompañamiento terapéutico podría ofrecer una operación denominada **ConsultarEncuentro**. Esta operación permitiría solicitar los datos de un encuentro con el paciente a partir de su identificador.
Para realizar la consulta, el cliente enviará un mensaje de entrada que contenga el identificador solicitado. El servicio procesa la solicitud y, si todo sale bien, devuelve un mensaje de salida con los datos correspondientes.

Mensajes

Los mensajes describen la información lógica que se intercambia entre el cliente y el proveedor del servicio.
Un mensaje puede representar una solicitud enviada al servidor o una respuesta que el servidor devuelve al cliente. Los elementos **message** permiten definir estos intercambios y sus partes, que se vinculan con elementos o tipos de datos.
Siguiendo con el ejemplo anterior.

- **ConsultarEncuentroInput** → Representa la información enviada al servicio para realizar la consulta. Contiene el identificador del encuentro que se quiere recuperar.

- **ConsultarEncuentroOutput** → Representa la información que devuelve el servicio como respuesta a la solicitud.

Tipos de datos

Los tipos de datos establecen cómo deben representarse los valores que aparecen en los mensajes. Estas definiciones suelen expresarse mediantes XML Schema.
La operación **ConsultarEncuentro** puede contemplar los siguientes campos.

Campo	Tipo de dato	Descripción
encuentroId	xsd:int	Identificador del encuentro que se quiere consultar
fecha	xsd:date	Fecha asociada al encuentro
descripción	xsd:string	Descripción breve de lo sucedido

XSD también permite definir estructuras más complejas, campos obligatorios u opcionales y restricciones adicionales sobre los valores admitidos.

Endpoints

Un endpoint es un punto de acceso concreto a través del cual puede invocarse un servicio. Se define mediante un elemento **port**, que asocia una dirección de red con un **binding**.

El binding especifica cómo se concreta la comunicación (qué protocolo y formato de mensajes se utilizarán). El endpoint indica dónde se encuentra disponible el servicio mediante una dirección. Para el proyecto, podría usarse la siguiente dirección (es de ejemplo, no es la real).
https://api.ejemplo.org/acompañamiento/soap

UDDI (Universal Description, Discovery and Integration)

Es un estándar que define un mecanismo para publicar, buscar y descubrir servicios web. Permite mantener un registro con información acerca de las organizaciones que ofrecen servicios, las funcionalidades que proporcionan y los datos técnicos necesarios para acceder a ellos.

Su fundamento principal consiste en facilitar que una aplicación encuentre un servicio adecuado sin necesitar conocer previamente todos los detalles de su proveedor o de su implementación. Para lograrlo, ofrece un mecanismo de registro

que permite organizar y consultar información sobre los servicios disponibles. En este registro, los proveedores pueden publicar información sobre los servicios que ofrecen, mientras que los consumidores pueden realizar búsquedas y recuperar las referencias necesarias para utilizarlos. En una integración basada en SOAP, esto puede incluir una referencia al documento WSDL.

Modelo de datos

UDDI define diferentes estructuras de información para organizar los datos de los proveedores y sus servicios.

- BusinessEntity → Organización proveedora de uno o más servicios.
- BusinessService → Servicio ofrecido por una organización registrada.
- BindingTemplate → Representa la información técnica para acceder a una instancia del servicio proporcionado.
- tModel → Es una estructura que permite identificar y referenciar especificaciones técnicas utilizadas para describir un servicio.

En conjunto, estas entidades permiten organizar la información necesaria para identificar una organización, conocer los servicios que ofrece y encontrar las referencias técnicas que facilitan su utilización.

Relación entre UDDI, WSDL y SOAP

Las tres son tecnologías complementarias dentro de la arquitectura tradicional de servicios web. Cada una cumple una función diferente en el proceso de integración entre aplicaciones.

Tecnología	Función principal
UDDI	Permite localizar un servicio y obtener sus referencias técnicas
WSDL	Específica sus operaciones, mensajes, tipos de datos y puntos de acceso
SOAP	Transporta las solicitudes y respuestas de las operaciones mediante mensajes SOAP

Tomemos como ejemplo una aplicación que necesita consultar información de un registro administrado por otro sistema. El proveedor pública información sobre su servicio en un registro UDDI. Cuando el consumidor encuentra el servicio, obtiene la referencia a su descripción WSDL. A partir de esa descripción, conoce qué operación debe invocar y qué estructura debe tener la solicitud. Por último, envía el mensaje SOAP al endpoint correspondiente y recibe una respuesta del proveedor. Además de servir como registro de servicios, UDDI define interfaces para realizar operaciones de publicación y consulta. Habitualmente, dichas interfaces pueden utilizar mensajes SOAP para comunicarse con el registro a través de HTTP.
Es decir, SOAP no solo se puede usar para consumir un servicio registrado, sino también para efectuar consultas o publicaciones en el registro UDDI.

Aspectos de seguridad del protocolo

La seguridad constituye un aspecto fundamental en el desarrollo y la utilización de servicios web, especialmente cuando las aplicaciones intercambian información sensible o necesitan garantizar que las operaciones sean realizadas únicamente por usuarios o sistemas autorizados.

SOAP permite estructurar e intercambiar mensajes entre aplicaciones de manera estandarizada. Sin embargo, no garantiza automáticamente la confidencialidad, la integridad ni el acceso autorizado a la información. Para proteger adecuadamente un servicio, es necesario combinar mecanismos de seguridad en las comunicaciones, estándares específicos para la protección de mensajes y controles implementados en la aplicación.

Los objetivos de seguridad que deben alcanzarse en un ecosistema SOAP son los siguientes.

- Confidencialidad → Consiste en garantizar que la información sólo pueda ser accedida por quienes tienen la autorización necesaria.

- Integridad → Consiste en garantizar que la información no haya sido modificada de manera no autorizada.

- Autenticación → Es el proceso mediante el cual se verifica la identidad de una persona o sistema que intenta acceder a un servicio.

- Autorización → Determina qué operaciones puede realizar un usuario o sistema autenticado y a qué recursos puede acceder.

- Disponibilidad → Consiste en garantizar que el servicio pueda utilizarse cuando sea necesario y que no quede inutilizado por fallos.

Estos objetivos pueden alcanzarse mediante distintos mecanismos. Algunos actúan sobre el canal de comunicación; otros protegen el contenido del mensaje; y otros deben implementarse en la lógica del servicio.

WS-Security

Es un conjunto de extensiones estandarizadas para incorporar mecanismos de seguridad a los mensajes SOAP. Su objetivo es permitir que la información de autenticación y protección criptográfica pueda asociarse directamente con los mensajes intercambiados.

Sus principales capacidades son las siguientes.

- Incorporar tokens de seguridad que permitan aportar información sobre la identidad del emisor del mensaje enviado.

- Utilizar firmas digitales para proteger la integridad de determinadas partes del mensaje, como el Body o el Header.

- Cifrar el cuerpo del mensaje para preservar su confidencialidad.

- Integrar información temporal que ayude a verificar la vigencia del mensaje.

Estas capacidades pueden utilizarse individualmente o en conjunto, según los requisitos de la aplicación.

Por ejemplo, una solicitud SOAP destinada a consultar un registro podría contener información de seguridad en su elemento Header. El servicio procesa esa información, verifica la identidad del solicitante y comprueba que la solicitud cumple con las condiciones de seguridad definidas.

WS-Security no determina por sí solo qué persona puede acceder a un registro concreto. La aplicación en cuestión debe interpretar la identidad y aplicar sus reglas de autorización.

HTTPS y TLS

HTTPS permite utilizar HTTP sobre una conexión protegida mediante TLS.

TLS ofrece mecanismos de confidencialidad e integridad para los datos transmitidos, y permite autenticar al servidor mediante certificados digitales. La autenticación del cliente se puede establecer cuando se configura TLS mutuo.

Para utilizar estos mecanismos correctamente, el consumidor debe verificar el certificado del servidor, comprobar que corresponda al dominio esperado y rechazar certificados inválidos o no confiables.

Utilizar HTTPS constituye una medida fundamental para un servicio SOAP expuesto a través de una red. Sin embargo, TLS no decide qué usuarios pueden consultar determinados registros ni reemplaza los controles de autorización de la aplicación.

Firma digital XML Signature

La firma digital permite verificar la integridad de los datos firmados y comprobar que fueron firmados mediante la clave privada correspondiente al certificado o a la clave pública utilizada para validar la firma.

WS-Security puede utilizar XML Signature para firmar el cuerpo de un mensaje SOAP, determinados elementos de su cabecera u otras partes seleccionadas.

El proceso consiste en generar una firma sobre el contenido definido y adjuntarla al mensaje. Al recibirlo, el destinatario verifica la firma y comprueba si el contenido protegido sigue siendo consistente con la firma original.

Cifrado mediante XML Encryption

Permite cifrar datos representados en XML y expresar el resultado dentro de una estructura con el mismo formato.

A través de WS-Security, puede aplicarse el cifrado al cuerpo de un mensaje SOAP o a partes específicas que contengan información confidencial.

Por ejemplo, si un mensaje contiene información que solo debe conocer el sistema destinatario, podría cifrarse el elemento que corresponda para impedir que intermediarios sin acceso a las claves puedan interpretar contenido.

Es distinta a la firma digital. Esta última se destina a detectar modificaciones y verificar la procedencia de los datos firmados. Mientras que el cifrado busca impedir que terceros no autorizados conozcan el contenido protegido.

Comparativa entre SOAP y REST

SOAP (Simple Object Access Protocol) y REST (Representational State Transfer) son dos enfoques utilizados para permitir la comunicación y el intercambio de información a través de una red. Se diferencian en su naturaleza, arquitectura y la forma en que organizan las solicitudes y las respuestas.

SOAP es un **protocolo de mensajería** que define una estructura para intercambiar información mediante mensajes. Puede complementarse con otras especificaciones para describir las operaciones disponibles, establecer contratos y abordar necesidades de seguridad y confiabilidad.

REST, en cambio, es un **estilo de arquitectura** para diseñar sistemas distribuidos. Se basa en la identificación de recursos, la transferencia de representaciones de esos recursos y una interfaz uniforme entre cliente y servidor. Aunque puede utilizar diferentes formatos de datos, suele implementarse mediante HTTP y utilizar JSON para representar la información intercambiada.

Es decir, SOAP define reglas concretas para la estructura de los mensajes, mientras que REST establece principios arquitectónicos para organizar la comunicación entre los sistemas.

Estándar de lenguaje SOAP

SOAP utiliza XML (Extensible Markup Language) para representar los mensajes. XML permite estructurar la información mediante etiquetas y establece una sintaxis común que facilita el procesamiento de los datos por diferentes aplicaciones.

Los mensajes SOAP tienen una estructura definida por el protocolo, que incluye una envoltura (Envelope), un cuerpo (Body) y una cabecera opcional (Header). Esta organización hace que la información respete un formato establecido.

SOAP se puede relacionar con el control estándares varias especificaciones.

- WSDL (Web Services Description Language) → Descripción formal del servicio, de las operaciones disponibles, los mensajes, los tipos de datos y los puntos de acceso.
- XML Schema (XSD) → Permite definir los tipos de datos y las estructuras que deben respetar los mensajes XML.
- WS-Security → Incorpora mecanismos para proteger mensajes mediante firmas digitales y cifrado.
- XML Signature y XML Encryption → Especificaciones relacionadas con la firma digital y el cifrado de contenidos XML.

Es importante destacar que no todos los mecanismos mencionados son obligatorios en cualquier implementación, se eligen de acuerdo con las características y los requisitos del servicio.

Estándar de lenguaje REST

REST no establece un único lenguaje obligatorio para representar los datos. Su arquitectura permite transferir diferentes representaciones de los recursos, siempre que los componentes respeten las reglas de interacción correspondientes.

En las API REST que utilizan HTTP, las solicitudes se realizan habitualmente mediante métodos estandarizados como **GET**, **POST**, **PUT** y **DELETE**. Los recursos se identifican mediante URI (Uniform Resource Identifier), que permiten determinar qué elemento se quiere consultar o modificar.
Los formatos de representación más habituales incluyen los siguientes.

- JSON (JavaScript Object Notation) → Formato de texto ligero que permite representar objetos mediante una estructura sencilla.
- XML → También puede usarse para representar datos en una API REST.

- HTML y texto plano → Pueden utilizarse cuando la aplicación necesita transferir contenido textual.

- Otros formatos → Según las necesidades, pueden emplearse imágenes, documentos u otros tipos de contenido.

JSON es el más frecuente en las API porque ofrece una representación compacta y sencilla de los datos, y resulta práctico para aplicaciones JavaScript que necesitan intercambiar información con un servidor.

¿REST más liviano que SOAP?

Se suele afirmar que REST es más liviano que SOAP porque muchas implementaciones REST utilizan mensajes HTTP sencillos y representaciones JSON, mientras que las de SOAP utilizan documentos XML siguiendo una estricta estructura y elementos adicionales.

La diferencia radica en la forma de estructurar los mensajes, el volumen de información transmitida y la complejidad de los mecanismos para describir y procesar las solicitudes.

- Menor cantidad de información estructural → En SOAP, hasta una operación sencilla necesita una estructura de mensaje que respete el protocolo. La envoltura XML, espacios de nombres, etiquetas, etc.
  En REST, una solicitud sencilla puede identificar el recurso directamente mediante la URI y utilizar el método HTTP apropiado.

- Menor complejidad de implementación → SOAP dispone de un conjunto de especificaciones para el contrato del servicio, la estructura de los mensajes y extensiones para seguridad.
  Una API REST puede implementarse utilizando las funcionalidades habituales de HTTP, rutas para identificar recursos y el formato JSON.

- JSON frente a XML → Este es uno de los motivos principales por los que las API REST suelen asociarse con una menor sobrecarga de datos. XML es un formato de marcado que incorpora etiquetas explícitas para estructurar los valores. JSON, utiliza pares de nombres y valores, arreglos y tipos de datos primitivos de una forma generalmente más compacta.

Cabe destacar que si bien se puede confirmar que “REST es más liviano que SOAP” en varios aspectos, siempre depende de lo que se desarrolle y el nivel de formalidad que se necesite.

Estándares más utilizados

Aspecto	SOAP	REST
Formato de mensajes habitual	XML estructurado	JSON
Estructura de la solicitud	Mensaje SOAP con una envoltura y un cuerpo	Método HTTP, URI, encabezados y, cuando corresponde, cuerpo
Descripción del servicio	WSDL y XML Schema (XSD)	OpenAPI suele ser una opción común para documentar
Comunicación habitual	HTTP y HTTPS	HTTP y HTTPS
Complejidad inicial	Mayor configuración y herramientas específicas	Puede ser menor, especialmente con JSON y solicitudes sin cuerpo
Seguridad	WS-Security, XML Signature, XML Encryption	TLS, JWT, API Keys

Ejemplo de caso de consumo

Facturación Electrónica de ARCA (WSFEv1): Un ejemplo práctico de amplio alcance en el que se emplea un servicio SOAP corresponde al Web Service de Factura Electrónica (WSFEv1) provisto por ARCA (ex AFIP) en la República Argentina. Los sistemas de gestión comercial que requieren la emisión de comprobantes válidos deben vincularse a este punto de acceso a fin de gestionar el Código de Autorización Electrónico (CAE).

Característica / Elemento	Identificación en el Web Service de ARCA
WSDL	ARCA expone un contrato público (service.asmx?WSDL) que define estrictamente las reglas, tipos de datos y operaciones admitidas para facturar.
Operaciones	Posee operaciones claras como FECAESolicitar (para pedir autorización de una factura) o FECompConsultar (para consultar un comprobante ya emitido).
Mensajes y Tipos de Datos	La solicitud de FECAESolicitar requiere un XML estructurado que incluye tipos de datos complejos como FECAEReq (encabezado con CUIT, Punto de Venta) y FECAEDetRequest (detalles de importes y tributos).
Seguridad (WS-Security)	No utiliza autenticación simple. Requiere un Token y un Sign generados previamente mediante un servicio de autenticación (WSAA) utilizando certificados digitales X.509 y firmas criptográficas (CMS).
Endpoints	Define URLs separadas: una para el entorno de Homologación (testing) y otra para el entorno de Producción.

Dominios de aplicación exclusiva de SOAP frente a REST

Si bien REST es el estándar predominante para el desarrollo de aplicaciones web y móviles modernas (como se evidencia en la arquitectura de RedAT), existen dominios dispares y críticos donde SOAP sigue siendo la elección prioritaria y, en muchos casos, obligatoria:

1. **Sector Bancario y Transacciones Financieras Complejas:** En los sistemas bancarios core o transferencias interbancarias internacionales, se requiere garantizar propiedades transaccionales ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) a través de múltiples sistemas distribuidos. SOAP permite esto gracias a su extensión **WS-Transaction**. Si un paso de la

transacción falla, el protocolo maneja el *rollback* de forma nativa e integrada, algo que en REST requeriría ser programado manualmente.

2. **Servicios Gubernamentales, Fiscales y de Salud (Interoperabilidad formal):** Sistemas como el mencionado de ARCA, o redes de intercambio de historias clínicas electrónicas (bajo normativas HL7), manejan datos altamente sensibles y requieren **no repudio**. Gracias a **WS-Security** y **XML Signature**, SOAP permite firmar digitalmente no solo el canal de transporte (como hace HTTPS en REST), sino el *cuerpo del mensaje en sí*. Esto asegura que el mensaje XML guarde validez legal y pueda auditarse, garantizando que nadie altere el contenido desde el momento en que salió del emisor hasta que llegó al receptor, independientemente de los intermediarios en la red.

La elección de SOAP en estos dominios se fundamenta en su capacidad para establecer **contratos estrictos** inquebrantables, su seguridad a nivel de mensaje (y no solo de transporte) y su fiabilidad transaccional estructurada por estándares globales, características que REST no ofrece de manera nativa al ser un estilo arquitectónico más flexible.

Integración de Servicios Web y APIs RESTful Externas

Para el desarrollo de la plataforma **RedAT**, se integraron dos APIs RESTful clave: la API de **Georef AR** y la API de **Cloudinary**. La elección de ambas herramientas responde al contexto del proyecto y a las restricciones operativas de los entornos de despliegue académicos.

1. API Geográfica de Argentina (Georef AR)

El objetivo de esta integración consiste en estandarizar la carga de datos en perfiles y búsquedas, facilitando la autocompletación de provincias y ciudades mediante registros oficiales de Argentina.

Criterio	Google Places API	Georef AR (Elegida)
Costo	Requiere método de pago / Tarifado	Gratuito (servicio público oficial)
Credenciales	Requiere API Keys y configuración	Sin necesidad de credenciales
Alcance	Global (requiere filtros para Argentina)	Nacional (oficial de Argentina)
Uso	Búsqueda general de lugares y comercios	Poblado consistente de entidades geográficas y localidades

La comunicación se realiza desde el backend en Node.js/Express aplicando un esquema Backend For Frontend. Se consumen las direcciones oficiales mediante fetch, filtrando atributos innecesarios antes de enviar los datos al cliente en React y registrar las referencias en la base de datos PostgreSQL mediante Prisma.

2. Almacenamiento en la Nube (Cloudinary REST API)

Permite gestionar la carga persistente de credenciales y certificados en formatos PDF e imagen, necesarios para validar a los profesionales en el sistema.

Se comparó el servicio con proveedores como **Amazon S3** y **Firebase Storage**.

Criterio	Amazon S3	Cloudinary (Elegida)
Acceso y Costo	Periodo de prueba requiere tarjeta de crédito	Opción gratuita sin requerimientos bancarios
Configuración	Requiere configuraciones complejas de permisos	Configuración sencilla e integración directa
Almacenamiento y Optimización	Almacenamiento de objetos estándar	Resuelve la pérdida de archivos en servidores efímeros y brinda optimización por URL para documentos e imágenes

Para la subida de archivos, se empleó el middleware Multer utilizando memoria RAM efímera. El servidor convierte el contenido a Base64 Data URI para enviarlo a la API de Cloudinary, almacenando posteriormente el enlace secure_url en el modelo DocumentAT.

Uso de IA

En el marco del desarrollo y las pruebas relativas a los servicios web, se empleó un modelo de Inteligencia Artificial (Gemini) como soporte para la consulta técnica y la resolución de fallos. En particular, la herramienta sirvió para identificar y corregir inconvenientes en la lógica del backend durante la integración de las interfaces de programación, abordando las siguientes tareas:

- **Diagnóstico de errores del servidor (HTTP 500)** → Se analizaron las trazas producidas al vincular la API de Cloudinary. La herramienta permitió detectar el origen de los fallos en el punto de acceso de registro, causados por el cambio desde el guardado en disco hacia el uso de memoria RAM (memory Storage). Por consiguiente, se modificó la lógica para transformar los ficheros en *Base64 Data URIs* antes de enviarlos, evitando inconsistencias en el ORM Prisma al almacenar el enlace del documento.

- **Resolución de conflictos transaccionales (HTTP 409)** → Se examinaron los errores de duplicidad al dar de alta usuarios en PostgreSQL, identificando datos parciales generados por inconvenientes en la distribución de correos.

Conclusión

El desarrollo de este trabajo práctico permitió consolidar los fundamentos teóricos sobre la interoperabilidad de sistemas, contrastando la flexibilidad de la arquitectura REST con la seguridad y robustez transaccional del protocolo SOAP. La aplicación práctica en la plataforma RedAT, a través de la integración de las APIs de Georef AR y Cloudinary, demostró cómo el consumo de servicios web externos optimiza la consistencia de los datos y la gestión de archivos multimedia sin sobrecargar la infraestructura propia. Asimismo, la incorporación de inteligencia artificial como soporte técnico resultó fundamental para diagnosticar errores de almacenamiento y resolver conflictos de base de datos en el backend, evidenciando el valor de complementar arquitecturas distribuidas con herramientas de asistencia modernas para garantizar un despliegue exitoso.